Resum Entorn

Josep Ferriol Font

27 de febrer de 2026

Capítol 1

Resum de l'Estructura

El projecte **TFG-truc** implementa el joc de cartes Truc amb una arquitectura modular preparada per a l'entrenament d'agents de Reinforcement Learning (RL).

Visió General

L'objectiu principal és proporcionar un entorn robust per simular partides de Truc i entrenar agents intel·ligents utilitzant la llibreria RLCard.

Estructura de Directoris

- `truc/`: Nucli del joc.
  - `game.py`: Motor lògic del joc. Gestiona els estats de la partida, les regles, la progressió de rondes i mans, i les apostes (Truc i Envit).
  - `env.py`: Adaptador per a RLCard. Tradueix l'estat intern del joc a observacions numèriques i exposa l'API estàndard de l'entorn (`reset`, `step`).
  - `cartes_accions.py`: Fitxer de constants compartides. Defineix la baralla espanyola de 36 cartes (4 pals $\times$ 9 rangs, sense els 2, 8 i 9), l'**espai d'accions** complet (18 accions: 3 jugar carta, 7 de aposta/pas i 8 de senyals) i les estructures `ACTION_LIST` i `ACTION_SPACE` que permeten convertir entre noms d'acció i índexs numèrics.
  - `rols/`: Implementacions dels rols del joc.
    - `dealer.py`: Gestiona la baralla (creació, barreja i repartiment de cartes als jugadors).
    - `judger.py`: Conté la lògica d'arbitratge: determina el guanyador d'una ronda (per força de carta), d'una mà (per majoria de rondes guanyades amb desempats) i de l'envit (per puntuació de la mà).
    - `player/player.py`: Classe base abstracta que defineix la interfície comuna per a qualsevol tipus de jugador.
    - `player/player_random.py`: Agent aleatori que selecciona uniformement entre les accions legals disponibles; útil com a baseline per avaluar altres agents.
    - `player/player_huma.py`: Jugador humà que demana l'acció per consola en cada torn.
- `demo.py`: Script de demostració interactiu per jugar per consola. Permet configurar el nombre de jugadors, el tipus de cada un (Humà o Random), les senyes i la puntuació final.
- `doc/`: Documentació tècnica en Markdown i LaTeX.

Capítol 2

Documentació de `game.py`

2.1 Visió General

`TrucGame` és la classe central que implementa tota la lògica del joc del Truc. Gestiona l'estat complet de la partida: jugadors, cartes, apostes (Truc i Envit), rondes, i puntuació global. Actua com a **motor del joc**, independent de la interfície d'entorn (`env.py`).

2.2 Constructor `__init__`

```
TrucGame(num_jugadors=2, cartes_jugador=3, senyes=False, puntuacio_final=24,
    player_class=TrucPlayer)
```

2.2.1 Paràmetres de configuració

Paràmetre	Defecte	Descripció
`num_jugadors`	2	Nombre de jugadors
`cartes_jugador`	3	Cartes repartides per mà
`senyes`	False	Activa la fase de senyals
`puntuacio_final`	24	Punts per guanyar la partida
`player_class`	`TrucPlayer`	Classe o **diccionari** `{id: Classe}` dels jugadors (permet partides mixtes Human vs Bot)

2.3 Variables Internes del Joc

2.3.1 Jugadors i Rols

Variable	Tipus	Descripció
`players`	`list[TrucPlayer]`	Llista d'instàncies de jugadors
`dealer`	`TrucDealer`	Gestiona la baralla i el repartiment
`judger`	`TrucJudger`	Determina guanyadors de rondes, envits i mans
`np_random`	`RandomState`	Generador aleatori per reproducibilitat

2.3.2 Control del Torn

Variable	Tipus	Descripció
`ma`	`int`	ID del jugador que és **mà** (comença jugant). Rota cada mà nova.
`current_player`	`int`	ID del jugador que ha d'actuar ara
`turn_player`	`int`	Jugador amb el torn "real" (es restaura després d'una aposta)
`turn_phase`	`int`	Fase actual del torn: 0=Senyals, 1=Apostes, 2=Jugar carta
`response_state`	`ResponseState`	Estat de resposta pendent

Valor	Significat
`NO_PENDING` (0)	No hi ha cap aposta pendent de resposta
`TRUC_PENDING` (1)	S'ha cantat Truc i s'espera resposta (Vull / Fora / Re-apostar)
`ENVIT_PENDING` (2)	S'ha cantat Envit i s'espera resposta (Vull / Fora / Re-apostar)

Important: Mentre hi ha una resposta pendent, el jugador contrari **només** pot respondre a l'aposta (vull/fora/re-apostar). No pot jugar cartes ni fer altres accions.

2.3.3 Puntuació i Historial

Variable	Tipus	Descripció
`score`	`list[int]`	Puntuació global `[equip_0, equip_1]`
`hist_cartes`	`list[tuple]`	Historial de cartes jugades: `(player_id, carta)`
`hist_senyes`	`list[tuple]`	Historial de senyals: `(player_id, senya)`
`round_counter`	`int`	Nombre de rondes completades a la mà actual
`cartes_ronda`	`list[tuple]`	Cartes jugades a la ronda en curs
`ronda_winners`	`list[int]`	Guanyadors de cada ronda (-1 = empat)
`payoffs`	`list[int]`	Recompenses finals per a l'entorn RL

2.3.4 Estat del Truc (Aposta de cartes)

Variable	Tipus	Descripció
`truc_level`	`int`	Nivell actual del Truc (punts en joc per cartes)
`truc_owner`	`int`	Qui ha cantat l'última aposta de Truc (-1 = ningú)
`previous_truc_level`	`int`	Nivell anterior (per si diuen "Fora", saben quants punts guanya el rival)

Seqüència d'escalada del Truc:

`1 (per defecte) -> 3 (Truc) -> 6 (Retruc) -> 9 (Val Nou) -> 24 (Joc Fora)`

2.3.5 Estat de l'Envit (Aposta d'envit)

Variable	Tipus	Descripció
`envit_level`	`int`	Nivell actual de l'Envit
`envit_owner`	`int`	Qui ha cantat l'últim Envit (-1 = ningú)
`envit_accepted`	`bool`	Si l'envit ha estat acceptat
`previous_envit_level`	`int`	Nivell anterior (per calcular punts si diuen "Fora")

Seqüència d'escalada de l'Envit:

`0 (no cantat) -> 2 (Envit) -> 4 (Un mes) -> 6 (Dos mes) -> Tots (Falta Envit)`

Regla "Tots" (Falta Envit): Si cap equip supera els 12 punts, val 24 (guanya partida). Si algun equip supera els 12, val `24 - puntuacio_del_lider`.

2.4 Mètodes Principals

2.4.1 `init_game()` → Inicialitzar Partida

1. Crea els jugadors amb `player_class`
2. Crea el dealer i el judger
3. Barreja i reparteix cartes
4. Inicialitza totes les variables d'estat
5. Retorna `(estat, current_player)`

2.4.2 `step(action)` → Avançar el Joc

El mètode principal. Rep una acció (int o string) i actualitza l'estat.

Flux de decisions:

```
step(action)
      |
      +--[Resposta pendent?]
      |        |
      |   ENVIT_PENDING --> vull_envit | fora_envit | apostar_envit
      |   TRUC_PENDING  --> vull_truc  | fora_truc  | apostar_truc
      |
      +--[NO_PENDING - Torn normal]
               |
               +-- Fase 0 (Senyals) --> passar  (avanca a Fase 1)
               |                    --> senya_* (registra, avanca a Fase 1)
               |
               +-- Fase 1 (Apostes) --> apostar_envit --> ENVIT_PENDING
               |                    --> apostar_truc  --> TRUC_PENDING
               |                    --> fora_truc     --> Rival guanya, reset ma
               |                    --> passar        --> avanca a Fase 2
               |
               +-- Fase 2 (Jugar)   --> play_card_X
                                         |
                                    Fi ronda? -> Fi ma? -> Fi partida?
```

2.4.3 `_get_return_state()` → Retorn Intel·ligent

Comprova si l'única acció legal és `passar`. Si és així, **executa el `passar` automàticament** per evitar torns trivials (important per a l'entrenament d'agents RL).

2.4.4 `get_state(player_id)` → Construir Estat

Retorna un diccionari amb tota la informació visible pel jugador:

2.4.5 `get_legal_actions()` → Accions Legals

Determina quines accions pot fer el jugador actual segons la situació:

Situació	Accions disponibles
`ENVIT_PENDING`	`vull_envit`, `fora_envit`, (+ `apostar_envit` si `level` $\leq$ `6`)
`TRUC_PENDING`	`vull_truc`, `fora_truc`, (+ `apostar_truc` si `level < 24`)
Fase 0 (Senyals)	`passar` + totes les `senya_*`
Fase 1 (Apostes)	`apostar_envit`[1], `apostar_truc`[2], `fora_truc`, `passar`
Fase 2 (Jugar)	`play_card_0`, `play_card_1`, ... (segons cartes disponibles)

[1] Només si `envit_level == 0` i `round_counter == 0` (primera ronda, sense envit previ)
[2] Només si el jugador actual no és el propietari de l'última aposta de Truc

2.4.6 `_reset_hand_state()` → Reset per Nova Mà

Quan acaba una mà (per guanyador o per "Fora"):

1. Avança `ma` al següent jugador
2. Barreja i reparteix de nou
3. Reseteja totes les variables de Truc, Envit, rondes i historials

2.4.7 `is_ma_over()` i `is_over()` → Fi de Mà / Partida

Distingeix els dos nivells d'acabament:

- `is_ma_over()`: retorna `True` si la **mà actual** ha acabat (hi ha guanyador o s'han esgotat les rondes).
- `is_over()`: retorna `True` si la **partida** ha acabat (`max(score) >= puntuacio_final`).

2.5 Casuístiques Especials

2.5.1 Determinar guanyador de la mà

El `TrucJudger.guanyador_ma()` segueix aquesta lògica:

1. Si un equip guanya la **majoria** de rondes ($\geq$2 de 3) → guanya
2. Si s'han jugat totes les rondes sense majoria:
   - L'equip amb més victòries guanya
   - Si estan igualats: guanya qui va guanyar la **primera ronda no empatada**
   - Si tot són empats: guanya el **mà**

2.5.2 Passar automàtic

`_get_return_state()` detecta quan l'única acció legal és `passar` i l'executa automàticament. Això evita que un agent RL hagi de "decidirüna acció trivial, millorant l'eficiència de l'entrenament.

2.5.3 Fases del torn

```
Fase 0: Senyals
    |  passar / senya_*
    v
Fase 1: Apostes
    |  passar
    v
Fase 2: Jugar Carta
    |  play_card_X
    v
Nova Ronda (torna a Fase 0 si senyes=True, sino Fase 1)
```

Nota: Si `senyes=False`, la fase 0 se salta completament i es comença directament a la fase 1.

Capítol 3

Documentació de `env.py`

3.1 Visió General

`TrucEnv` és l'entorn que adapta la lògica del joc (`TrucGame`) a la interfície estàndard de **RLCard**. La seva funció principal és **transformar l'estat del joc** (diccionaris llegibles per humans) en **vectors numèrics** (observacions) que un agent de Reinforcement Learning pot processar.

3.2 Arquitectura

```
                    observació, acció
  +------------+  <------------------>  +------------+
  |  Agent RL  |                        |  TrucEnv   |
  +------------+                        |  (env.py)  |
                                        +-----+------+
                                              | estat, step
                                              v
                                        +------------+
                                        |  TrucGame  |
                                        |  (game.py) |
                                        +--+--+--+---+
                                           |  |  |
                                 +------------+  |  +------------+
                                 v               v               v
                           +----------+   +-----------+   +-----------+
                           |TrucDealer|   |TrucJudger |   |TrucPlayer |
                           +----------+   +-----------+   +-----------+
```

`TrucEnv` hereta de `rlcard.envs.Env`, que proporciona el bucle estàndard `reset()` → `step()` → `get_payoffs()`.

3.3 Constructor `__init__`

`TrucEnv(config)`

3.3.1 Paràmetres de configuració (diccionari `config`)

Clau	Defecte	Descripció
`num_jugadors`	2	Nombre de jugadors
`cartes_jugador`	3	Cartes per jugador
`puntuacio_final`	24	Punts per guanyar
`senyes`	False	Activar fase de senyals
`player_class`	None	Classe dels jugadors (e.g. `HumanPlayer`) o **diccionari** `{id: Classe}` per barrejar tipus
`allow_step_back`	False	Permetre desfer passos (heretat d'RLCard)
`seed`	-	Seed per reproducibilitat

3.4 Variables Internes de l'Entorn

3.4.1 Mapeig de Cartes i Senyals

Variable	Tipus	Descripció
`cartes`	`list[str]`	Llista de totes les cartes del joc (36 cartes)
`carta_map`	`dict[str, int]`	Mapa `carta` → `index` per codificar cartes a one-hot
`signal_map`	`dict[str, int]`	Mapa `senya` → `index` per codificar senyals a one-hot

3.4.2 Dimensions de l'Espai d'Estat

Variable	Valor (exemple 2J, 3C, -senyes)	Descripció
`num_cartes`	36	Cartes úniques a la baralla
`espai_joc_cartes`	37	`num_cartes + 1` (inclou slot "buit")
`espai_hist_cartes`	6	`num_jugadors` × `cartes_jugador` (slots historial)
`espai_senya`	10	`len(ACTIONS_SIGNAL) + 1` (9 senyals + buit)
`espai_hist_senyes`	0 o 6	0 si `senyes=False`, sino `num_jugadors` × `cartes_jugador`
`espai_info_publica`	10	Puntuacions + apostes + situació
`state_size`	**343** (sense senyes)	Mida total del vector d'observació

3.4.3 Espais per a RLCard

Variable	Descripció
`state_shape`	`[[state_size]] * num_jugadors` — dimensions de l'observació per jugador
`action_shape`	`[[len(ACTION_LIST)]] * num_jugadors` — 18 accions possibles

3.5 Mètode Principal: `_extract_state(state)`

Transforma el diccionari d'estat del joc en un vector numèric `obs` de tipus `np.int8`. L'estructura del vector és:

3.5.1 Estructura del Vector d'Observació

```
+-------------------------------------------------------------------------+
|                    VECTOR D'OBSERVACIÓ (obs)                            |
+----------------------+--------------------------------------------------+
| SECCIÓ               | MIDA (exemple: 2J, 3C, sense senyes)             |
+----------------------+--------------------------------------------------+
| Mà del jugador       | 3 x 37 = 111  (3 slots one-hot de 37 pos.)       |
| Historial cartes     | 6 x 37 = 222  (6 slots one-hot de 37 pos.)       |
| Historial senyals    | 0 (desactivat) o 6 x 10 = 60                     |
| Info pública         | 10 valors escalars                               |
+----------------------+--------------------------------------------------+
| TOTAL                | 343 (sense senyes) o 403 (amb senyes)            |
+----------------------+--------------------------------------------------+
```

3.5.2 Detall de cada secció

1. Mà del Jugador (one-hot × `cartes_jugador`)

Per cada slot de carta a la mà:

- Si la carta existeix → marca `1` a la posició de la carta dins dels 36 valors
- Si el slot està buit (ja s'ha jugat) → marca `1` a la posició 37 (simbol "buit")

2. Historial de Cartes (one-hot × `espai_hist_cartes`)

Codifica cada carta jugada durant la mà. Mateixa codificació que la mà: one-hot de 37 posicions. Permet a l'agent recordar quines cartes s'han jugat i per qui.

3. Historial de Senyals (one-hot × `espai_hist_senyes`)

Només actiu si `senyes=True`. Per cada slot de senyal:

- Si existeix una senya → marca `1` a la posició corresponent (0-8)
- Si el slot està buit → marca `1` a la posició 10 (simbol "buit")

4. Informació Pública (10 valors escalars)

Posició	Contingut	Descripció
0	`puntuacio[0]`	Punts de l'equip 0
1	`puntuacio[1]`	Punts de l'equip 1
2	`estat_truc.level`	Nivell actual del Truc (1,3,6,9,24)
3	`estat_truc.owner + 1`	Qui ha cantat Truc (0=Ningú, 1=J0, 2=J1)
4	`estat_envit.level`	Nivell actual de l'Envit (0,2,4,6,...)
5	`estat_envit.owner + 1`	Qui ha cantat Envit (0=Ningú, 1=J0, 2=J1)
6	`fase_torn`	Fase actual (0/1/2)
7	`ma`	Qui és mà

8	comptador_ronda	Nombre de rondes completades
9	id_jugador	ID del jugador que observa

3.5.3 Retorn de _extract_state

```
{
    'obs': np.array([...]),        # Vector numeric per a la xarxa neuronal
    'legal_actions': OrderedDict,  # Accions legals com a OrderedDict
    'raw_obs': state,              # Estat original (diccionari llegible)
    'raw_legal_actions': ['passar', 'apostar_truc', ...], # Noms de les accions
    'action_record': self.action_recorder # Historial d'accions (d'RLCard)
}
```

3.6 Altres Mètodes

3.6.1 get_payoffs() → Recompenses Finals

Delega a `TrucGame.get_payoffs()`, que retorna la diferència de puntuació:

```
[score[0] - score[1], score[1] - score[0]]
```

L'agent que guanyi tindrà payoff positiu, el perdedor negatiu.

3.6.2 _decode_action(action_id) → Descodificar Acció

Converteix un índex d'acció numèric al seu nom string:

```
ACTION_LIST[action_id] # e.g. 0 -> 'play_card_0', 4 -> 'apostar_truc'
```

3.6.3 _get_legal_actions() → Accions Legals

Aquest mètode de l'entorn delega directament a `TrucGame.get_legal_actions()`, que és on realment es calcula la llista d'accions permeses. Retorna una **llista d'índexs** (enters) que representen les accions vàlides.

3.7 Espai d'Accions Complet

Definit a `cartes_accions.py`:

Índex	Acció	Categoria
0	play_card_0	Jugar carta
1	play_card_1	Jugar carta
2	play_card_2	Jugar carta
3	apostar_envit	Aposta
4	apostar_truc	Aposta
5	vull_envit	Resposta
6	vull_truc	Resposta
7	fora_envit	Resposta
8	fora_truc	Resposta/Retirada
9	passar	Control de flux

10–18	`senya_*`	Senyals (9 tipus)

3.8 Flux d'Ús Típic

```
# 1. Configurar
config = {'num_jugadors': 2, 'cartes_jugador': 3, 'senyes': False, ...}
env = TrucEnv(config)

# 2. Inicialitzar partida
state, player_id = env.reset()

# 3. Bucle del joc
while not env.is_over():
    action = agent.triar_accio(state) # L'agent escull
    state, player_id = env.step(action)

# 4. Obtenir recompenses
payoffs = env.get_payoffs() # [+diff, -diff]
```

3.9 Casuístiques Importants

3.9.1 Informació Asimètrica

Cada jugador només veu les seves pròpies cartes a `ma_jugador`. Les cartes dels rivals **no** s'inclouen al vector d'observació. L'historial de cartes jugades (`hist_cartes`) sí que és públic.

3.9.2 Codificació One-Hot

S'utilitza one-hot en lloc de valors ordinals perquè la xarxa neuronal no assumeixi una relació lineal entre les cartes. Cada carta és un vector binari independent.

3.9.3 Slot "Buit"

L'extra posició (`num_cartes + 1` = 37) serveix per marcar slots que encara no tenen carta. Sense aquest símbol, el vector no podria distingir "no s'ha jugat cap carta" de "s'ha jugat la carta 0".

Capítol 4

Arquitectura MVC

Aquest capítol descriu l'arquitectura Model–Vista–Controlador (MVC) del projecte: el paper del controlador i els contractes que ha de complir tant el model com la vista per poder-hi connectar-se.

4.1 Visió general

L'aplicació separa clarament tres rols:

- **Model**: lògica del joc (estat, regles, jugadors bots).
- **Vista**: entrada i sortida amb l'usuari (configuració, mostrat d'estat, selecció d'accions, resultats).
- **Controlador**: orquestra el flux; demana dades i accions a la vista, consulta i modifica l'estat a través del model, i no depèn de cap implementació concreta de vista o model.

El controlador depèn només d'**interfícies** (contractes): qualsevol vista i qualsevol model que implementin aquests contractes poden ser utilitzats sense canviar el controlador.

4.2 El controlador

El **controlador** (`controlador/controlador.py`) és l'únic punt que coneix tant la vista com el model. La seva funció és:

1. Obtenir la configuració inicial mitjançant la vista.
2. Inicialitzar el model amb aquesta configuració.
3. En un bucle, mentre la partida no hagi acabat:
   - Saber quin jugador juga (model).
   - Si és humà: mostrar l'estat (vista), demanar una acció (vista), aplicar-la (model) i informar la vista.
   - Si és bot: obtenir l'acció del model, aplicar-la i informar la vista.
4. Un cop acabada la partida, obtenir el resultat del model i mostrar-lo per la vista.
5. Preguntar si es vol repetir (vista) i, si cal, mostrar el missatge de sortida.

El controlador **no** conté lògica de joc ni lògica d'interfície: només coordina crides entre vista i model segons el contracte.

4.2.1 Flux del controlador

- **Inici**: `vista.demanar_config()` → `model.iniciar(config)`.
- **Bucle de partida** (fins `model.es_final()`):
  - `model.get_jugador_actual()`.
  - Si `model.es_huma(pid)`:
    * `model.get_estat(pid)` → `vista.mostrar_estat(estat)`.
    * `vista.escollir_accio(accions_legals, estat)` → `model.aplicar_accio(accio)`.
    * `vista.mostrar_accio(pid, nom_accio, es_bot=False)`.
  - Sinó (bot):
    * `model.get_accio_bot(pid)` → `model.aplicar_accio(accio)`.
    * `vista.mostrar_accio(pid, nom_accio, es_bot=True)`.
- **Fi partida**: `model.get_resultat()` → `vista.mostrar_fi_partida(score, payoffs)`.
- **Repetir**: `vista.demanar_repetir()`; si no, `vista.mostrar_sortint()`.

4.3 Contracte amb el model

El controlador parla amb el model a través del **protocol** `Model` (`controlador/interficie_model.py`). Qualsevol classe que implementi aquests mètodes pot fer de model.

Mètode	Signatura	Descripció
`iniciar`	`(self, config: dict) -> None`	Crea i inicialitza una partida amb la configuració donada.
`get_estat`	`(self, jugador_id: int) -> dict`	Retorna l'estat visible per al jugador amb aquest id.
`get_jugador_actual`	`(self) -> int`	Retorna l'id del jugador que ha de jugar ara.
`es_huma`	`(self, jugador_id: int) -> bool`	Indica si el jugador és humà (l'accions vindran de la vista).
`get_accio_bot`	`(self, jugador_id: int) -> tuple[int, str]`	Retorna `(codi_accio, nom_accio)` triada pel bot.
`aplicar_accio`	`(self, accio: int) -> None`	Aplica l'acció amb codi donat i avança l'estat del joc.
`es_final`	`(self) -> bool`	Indica si la partida ha acabat.
`get_resultat`	`(self) -> dict`	Retorna un diccionari amb `score` i `payoffs`.

La implementació actual del contracte `Model` és `ModelInteractiu` (`controlador/model_interactiu.py`), que encapsula el joc (`TrucGame`) i adapta les seves crides als mètodes del protocol.

4.4 Contracte amb la vista

El controlador parla amb la vista a través del **protocol** `Vista` (`vista/interficie_vista.py`). Qualsevol classe que implementi aquests mètodes pot fer de vista.

Mètode	Signatura	Descripció
`demanar_config`	`(self) -> dict`	Demana (per la UI) i retorna la configuració del joc.
`mostrar_estat`	`(self, estat: dict) -> None`	Mostra l'estat actual del joc (taula, mà, puntuació, etc.).
`escollir_accio`	`(self, accions_legals: list, estat: dict) -> int`	Presenta les accions legals; l'usuari en tria una; retorna el **codi** d'acció (enter).
`mostrar_accio`	`(self, jugador_id: int, nom_accio: str, es_bot: bool) -> None`	Informa quina acció ha fet un jugador. Si `es_bot=True`, la vista pot afegir retard visual.
`mostrar_fi_partida`	`(self, score: list, payoffs: list) -> None`	Mostra el resultat final (marcador i payoffs).
`demanar_repetir`	`(self) -> bool`	Pregunta si es vol jugar una altra partida. Retorna `True` si sí.
`mostrar_sortint`	`(self) -> None`	Indica que l'usuari surt de l'aplicació.

4.4.1 Aplicació del contracte a VistaConsola

La vista per consola (`vista/vista_consola.py`) implementa el protocol `Vista` mitjançant terminal: tot es mostra amb `print()` i es llegeix amb `input()`. Cada mètode del contracte es concreta així:

- **demanar_config**: retorna una còpia de la configuració per defecte (`CONFIG_PER_DEFECTE`), sense demanar res per teclat.
- **mostrar_estat**: imprimeix puntuació dels dos equipaments, número de ronda, jugador de la mà, nivell de truc i d'envit (si n'hi ha), les cartes a la taula i la mà del jugador.
- **escollir_accio**: numera les accions legals (0, 1, ...), imprimeix la llista amb el nom de cada acció (`ACTION_LIST`), llegeix un número amb `input()` i retorna el codi d'acció corresponent; en cas d'entrada invàlida o EOF/KeyboardInterrupt, es torna a demanar o es surt.
- **mostrar_accio**: imprimeix quin jugador ha jugat i el nom de l'acció ("Tu" o "Bot"); si `es_bot=True`, fa `time.sleep(BOT_DELAY_S)` per donar un retard visual.
- **mostrar_fi_partida**: imprimeix el marcador final i els payoffs.
- **demanar_repetir**: demana "s/n" per teclat i retorna `True` si la resposta és "s"; en EOF o KeyboardInterrupt retorna `False`.
- **mostrar_sortint**: imprimeix "Sortint...".

Així, el contracte `Vista` queda cobert per una implementació sense finestres ni servidor, útil per proves i ús en terminal.

4.5 Resum de l'MVC

- El **controlador** orquestra una partida: demana config i accions a la vista, i consulta/modifica l'estat mitjançant el model.
- El **contracte amb el model** (`Model`) defineix: iniciar partida, consultar estat i jugador actual, saber si és humà, obtenir acció del bot, aplicar acció, saber si és final i obtenir resultat.
- El **contracte amb la vista** (`Vista`) defineix: demanar config, mostrar estat, escollir acció (retornant codi), mostrar acció, mostrar fi de partida, demanar repetir i mostrar sortida.

Gràcies a aquests dos contractes, es poden afegir noves vistes o variants del model sense modificar el controlador, mantenint una arquitectura MVC clara i desacoblada.